1. Metaheuristic Framework and Solution Encoding

Since the allocation and scheduling problem formulated in Section **??** is NP-hard, a set of metaheuristic approaches were applied to explore the solution space for large-scale instances. In this context, a solution is represented through an explicit encoding that separates *allocation* decisions from *scheduling priority* decisions. This representation allows different search strategies to operate over a common candidate structure while delegating feasibility verification to the greedy heuristic defined previously.

1.1. Candidate Representation

As defined in section **??**, S denote the number of tasks and P the number of servers. A candidate solution c is defined as the tuple:

$$c = (\mathbf{p}, \mathbf{q}), \tag{1}$$

where:

- $\mathbf{p} = (p_1, \ldots, p_S)$ is the *server assignment vector* with $p_i \in \{1, \ldots, P\}$ indicating the server to which task τ_i is allocated.
- $\mathbf{q} = (q_1, \ldots, q_S)$ is the *priority vector*, with $q_i \in [0, 1]$ encoding the relative execution priority of task τ_i within its assigned server.

The priority vector does not directly encode start times. Instead, it induces an ordering among tasks assigned to the same processor. The actual schedule (starting times, finishing times, and feasibility verification) is obtained by invoking the scheduling procedure, which processes tasks in decreasing order of priority while enforcing all constraints, including precedence, deadlines and resource capacities of the servers.

This separation between encoding and feasibility evaluation enables the metaheuristics to operate in a continuous or discrete search space while maintaining strict feasibility control through the scheduler.

1.2. Initialization from a Greedy Precalculated Solution

All of the metaheuristics proposed in this paper require a random initialization. Generating a feasible schedule using complete random vectors p and q is an almost impossible task, even for small instances. For this reason, candidates are initialized from a feasible schedule previously computed using the greedy heuristic described in Section **??**.

This initial solution, not only works as metaheuristic methods starting point, but it also significantly improves the convergence behavior of the algorithms.

The greedy procedure exports a schedule, this is, for each task τ_i, an assigned server s_i^G, a start time t_i^G and a finishing time f_i^G. The superscript G denotes values obtained from the greedy solution. These values are used to construct initial candidates $c^{(0)}$ for the metaheuristic search. The initialization of the candidate $c^{(0)}$ is performed as follows.

For every task τ_i, the server index is directly inherited from the greedy solution:

$$s_i^{(0)} = s_i^G.$$

This guarantees that the initial candidate encodes a feasible allocation with respect to fixed MIST constraints, resource capacities, and compatibility rules.

Since the candidate encoding requires relative priorities rather than explicit start times, priorities are reconstructed from the greedy schedule.

Let π be a permutation of tasks such that, when ordered according to their start times in the greedy solution, they satisfy:

$$t^G_{\pi(1)} \leq t^G_{\pi(2)} \leq \cdots \leq t^G_{\pi(S)}.$$

Define the rank function $r(i)$ as the position of task τ_i in this ordered list (with $r = 0$ for the earliest task). Priorities are then assigned according to:

$$q_i^{(0)} = \frac{S - r(i)}{S}.$$

This normalization ensures $q_i^{(0)} \in (0, 1]$, where earlier tasks receive larger priority values. Consequently, when the scheduler reconstructs the execution order, it reproduces the greedy sequencing unless modified by later perturbations.

1.3. Search Space Interpretation

Under the encoding proposed in equation 1, the search space can be interpreted as a discrete combinatorial space induced by server assignments $\mathbf{s}$ and a continuous priority space $\mathbf{q} \in [0, 1]^S$ controlling intra-processor ordering.

Metaheuristic operators therefore act in two complementary ways. It can be by modifying elements of $\mathbf{s}$ for tasks without fixed assignments or by applying controlled stochastic variations to $\mathbf{q}$, thus altering execution ordering while preserving feasibility verification through the scheduler. This hybrid discrete–continuous structure provides sufficient flexibility to escape local minima while maintaining a compact and structured solution encoding.

1.4. Rationale for Greedy-Based Initialization

The greedy heuristic constructs solutions by explicitly enforcing feasibility at every assignment step. Initializing the metaheuristics from such a solution offers several advantages. For instance, it guarantees the existence of a feasible starting point, reduces the search effort required to reach feasible regions, provides a structured baseline reflecting precedence and deadline awareness and accelerates convergence in Simulated Annealing and Genetic Algorithms.

Subsequent metaheuristic exploration refines this initial solution by reallocating non-fixed tasks and adjusting priority values to improve the scalarized objective function defined in Section **??**. This approach effectively combines the strengths of constructive

heuristics with the global search capabilities of metaheuristics, leading to high-quality solutions for complex scheduling problems.

1.5. Algorithms Description

The metaheuristic strategies proposed in this work navigate the sparse feasible region of the search space. They utilize the indirect encoding defined by 1 that separates allocation decisions from task priorities. Feasibility is maintained via a deterministic scheduler that enforces all temporal and structural constraints, effectively decoupling the stochastic search process from feasibility control.

The neighborhood structure is governed by the `randomizeCandidate` operator, which was implemented using the Algorithm **??**. This algorithm perturbs solutions in two dimensions:

1. **Allocation:** Reassigning non-MIST tasks to admissible processors within the set of available resources.
2. **Priority:** Modifying execution order through additive Gaussian noise applied to continuous priority variables.

Algorithm 1 Operator `randomizeCandidate`(candidate, perturbationRate)

```
for i ← 1 to #tasks do
    if rand() < perturbationRate then
        if task i has no fixed assignment then
            assign candidate.server_indices[i] to a random non-MIST server
            continue
        end if
    end if
    if rand() < perturbationRate then
        candidate.priorities[i] ← candidate.priorities[i] + N(0, 0,05)
        clamp candidate.priorities[i] to the interval [0, 1]
    end if
end for
```

By verifying feasibility only after perturbation, the search is naturally biased toward valid regions; infeasible neighbors are discarded, ensuring that every accepted solution satisfies the system requirements.

The Random Search (RS) procedure serves as both a baseline and a crucial initialization tool. As it can be observed in Algorithm **??**, starting from a greedy-derived feasible solution, it applies controlled perturbations to explore the search space until a stagnation criterion or timeout is met. This ensures that subsequent metaheuristics begin within a feasible basin, which is essential given the low probability of generating valid schedules through purely random initialization.

Algorithm 2 Random Search

```
initialize limits, timeout, and stagnation counters
obtain initial candidate curr from the scheduler
bestFitness ← +∞
for iter ← 1 to rs_maxIterations do
    if timeout reached then
        break
    end if
    if schedule(curr) is feasible then
        if rs_breakOnFirstFeasible then
            return curr
        end if
        fitness ← f(curr)
        if fitness < bestFitness then
            update best and bestFitness
            reset stagnation counter
        else
            increment stagnation counter
            if counter ≥ rs_stagnationLimit then
                break
            end if
        end if
    end if
    randomizeCandidate(curr, rs_perturbationRate)
end for
if schedule(best) is feasible then
    return best
else
    return "no feasible solution"
end if
```

The Simulated Annealing (SA) algorithm introduces a probabilistic acceptance mechanism to escape local minima. Governed by a geometric cooling schedule, the search shifts from exploration to exploitation as the temperature decreases. Pseudocode of this methods is shown in Algorithm **??**.

Algorithm 3 Simulated Annealing

```
initialize with Random Search configured with breakOnFirstFeasible = true
if no feasible initial solution is found then
    return initialization error
end if
curr ← initial solution; best ← curr
currFitness ← f(curr); bestFitness ← currFitness
T ← sa_initialTemperature
for iter ← 1 to sa_maxIterations while T > sa_minTemperature do
    if timeout reached then
        break
    end if
    attempt up to sa_maxNeighborTries neighbors by perturbing curr
    if no feasible neighbor exists then
        T ← T · sa_coolingRate; continue
    end if
    if f(next) < f(curr) then
        accept next
    else
        accept with probability exp(−(f(next) − f(curr))/T)
    end if
    if accepted then
        curr ← next; currFitness ← f(next)
        apply refinement (Normal or PSO) on curr
        if currFitness improves bestFitness then
            update best and reset stagnation
        else
            increment stagnation; if it exceeds limit, terminate
        end if
    end if
    T ← T · sa_coolingRate
end for
return best
```

A key enhancement in this implementation is the inclusion of a local refinement phase after each accepted move. Two strategies are utilized: Normal refinement, which performs stochastic adjustments of the priority vector (Algorithm **??**), and PSO-based refinement, which treats the priority vector as a particle in a continuous domain (Algorithm **??**). Both methods intensify the search within the priority subspace while keeping server assignments fixed.

Algorithm 4 Normal Priority Refinement (SA)

```
σ ← máx(sa_sigmaMax · T/sa_initialTemperature, sa_sigmaMin)
noImprovementCounter ← 0
for iter ← 1 to sa_refinementIterations do
    trial ← copy of curr
    for each task i do
        trial.priorities[i] ← clamp(trial.priorities[i] + N(0, σ), 0, 1)
    end for
    if schedule(trial) is feasible and f(trial) < currFitness then
        curr ← trial; currFitness ← f(trial)
        noImprovementCounter ← 0
    else
        noImprovementCounter ← noImprovementCounter +1
    end if
    if noImprovementCounter ≥ 10 then
        break
    end if
end for
```

Algorithm 5 PSO Priority Refinement (SA)

```
initialize swarm of size sa_pso_swarmSize with random priorities and velocities
evaluate feasible particles to initialize personal bests
globalBest ← curr; globalBestFitness ← currFitness
v_max ← máx(sa_pso_velocityClamp · T/sa_initialTemperature, 0,1)
for iter ← 1 to sa_refinementIterations do
    for each particle p do
        for each task j do
            update velocity:
            v_j ← wv_j + c_1 r_1 (pbest_j − x_j) + c_2 r_2 (gbest_j − x_j)
            clamp v_j to [−v_max, v_max]
            x_j ← clamp(x_j + v_j, 0, 1)
        end for
        if schedule(position) is feasible then
            evaluate fitness and update pbest / gbest
        end if
    end for
end for
if globalBestFitness < currFitness then
    curr ← globalBest
end if
```

The Genetic Algorithm (GA) detailed in Algorithm **??** extends the exploration to

a population-based paradigm, initialized with feasible individuals from the RS phase. Selection is managed via a tournament mechanism to maintain competitive pressure, while elitism preserves high-quality solutions across generations. The crossover operator combines parent information by mixing server assignments and averaging priority values. Mutation is handled by the *randomizeCandidate* operator, and feasibility is strictly verified prior to population insertion.

Algorithm 6 Genetic Algorithm

```
initialize population of size ga_populationSize using feasible Random Search
if initialization insufficient/failed then
    return initialization error
end if
sort population by fitness and select global best
for gen ← 1 to ga_maxGenerations do
    if timeout reached then
        break
    end if
    new population ← best ga_eliteCount (elitism)
    while size(new population) < ga_populationSize do
        select parents by tournament (k=3)
        create child as copy of parent 1
        if rand() < ga_crossoverRate then
            for each task: mix server (50 %) and average priority
        end if
        mutate child with randomizeCandidate(child, ga_mutationRate)
        if schedule(child) is feasible then
            add child and its fitness to new population
        end if
    end while
    replace population and sort by fitness
    if current best improves global best then
        update global best and reset stagnation
    else
        increment stagnation; if it exceeds limit, terminate
    end if
end for
return global best individual
```

Overall, the proposed metaheuristic framework combines constructive feasibility, indirect encoding, stochastic exploration, and adaptive refinement within a unified architecture. Random Search provides diversification and initialization capability; Simulated Annealing offers controlled escape from local optima through temperature-guided acceptance; and the Genetic Algorithm introduces population-level recombination and long-

range exploration. The integration of these mechanisms enables efficient navigation of a highly constrained, hybrid discrete–continuous search space.